

Query-Aware Parallel JSON Processing: Fusing Compilation and Parsing in a Bytecode-VM Architecture

Problem Statement

JSON has become the dominant data interchange format, yet command-line JSON processors (jq, gojq, jaq) treat parsing and query execution as strictly separate phases. The parser materializes every field of every record into an intermediate representation before the query engine examines any of it. For selective queries — extracting one field from wide objects, filtering rare events from log streams — this design wastes 60–90% of parsing effort on data the query never touches. Prior work on query-aware parsing (Mison, VLDB 2017; Sparser, VLDB 2018; dsJSON, SIGMOD 2023) has demonstrated large gains, but exclusively in single-threaded research prototypes or SQL-oriented distributed systems. No existing system unifies query-aware parsing with a parallel execution engine driven by a bytecode-VM compiler — the architecture that practical JSON query tools actually use.

Proposed Contribution

This thesis designs, implements, and evaluates **compiler-driven query-aware parsing** in zq, a parallel JSON processor with a SIMD-accelerated tape-based parser and a bytecode-VM query engine. The core idea: the query compiler analyzes the filter program and emits a **parse plan** — a compact specification of which fields to materialize and which predicates to evaluate during parsing — that is passed to the parser before any data is read.

Three techniques are integrated into this architecture:

1. **Projection pushdown.** The compiler identifies accessed fields via its existing fuse pass. The parser builds structural indexes (SIMD colon/comma bitmaps) only for required nesting levels and skips tape construction for unneeded fields. Extends Mison’s approach to a multi-threaded chunk-parallel system where each worker receives the same parse plan.
2. **Predicate pushdown.** For `select(.field == value)` patterns, the compiler emits parse-and-filter plans. The parser evaluates the predicate as soon as the relevant field is encountered, rejecting records before constructing the remaining tape. Adapts dsJSON’s projection-tree concept to a bytecode-VM compilation target.
3. **Compiler-informed data structures.** Object key indexing (hash table for objects exceeding N keys), cached array/object lengths in the tape, and copy-on-write strings for unescaped values — each gated by compile-time analysis of which builtins the query invokes.

Evaluation

A secondary contribution is a **rigorous cross-tool benchmark suite** for JSON query processing — the first to systematically compare jq, jaq, gojq, zq, DuckDB `read_json`, and simdjson across controlled workload dimensions: record width (5–500 fields), nesting depth (1–20 levels), selectivity (0.1%–100%), file size (1 MB–100 GB), and schema homogeneity. The benchmark uses curated datasets from five domains (API responses, application logs, infrastructure configs, geospatial, scientific) and measures throughput, memory, and latency at core counts from 1 to 64.

Each optimization is evaluated independently (flag on/off) and in combination, with micro-benchmarks isolating parse, lookup, predicate, and serialization phases. This design attributes gains to specific techniques and identifies workload regimes where each optimization helps or hurts.

Expected Results

- **2–5x throughput improvement** on selective queries (`.id`, `select()`, `has()`) over wide objects, driven primarily by projection pushdown eliminating the current 73% parse-time bottleneck.
- **Workload characterization** showing that projection pushdown dominates on wide records, predicate pushdown dominates on high-selectivity filters, and the two compound multiplicatively on selective queries over wide data.
- **Cross-tool analysis** establishing performance profiles across the JSON query landscape, filling a gap in the existing literature where each prior system benchmarks only against its immediate predecessor.

Positioning

This work sits at the intersection of query compilation (Kersten et al., PVLDB 2018), SIMD-accelerated parsing (Langdale & Lemire, VLDB 2019), and query-aware data access (Mison; Sparser; dsJSON). The novelty is the integration point: a bytecode-VM compiler that drives a parallel SIMD parser — bridging the gap between research prototypes that demonstrate individual techniques and production tools that ignore them entirely.